

Hybrid AI 에이전트 시스템 접근법: 멀티 에이전트 환경에서의 신뢰성 및 효율성 증대

최남규, 김현준, 이해은, 전병욱, 박재일
이아이씨엔, KT gemie music/한양대학교,
한서대(태안군강소형스마트시티사업단)/아주대학교, 이아이씨엔, 아주대학교*

발표자 이메일: hans@eicn.co.kr

CONTENTS (1/2)

1. 서론

- 1.1 연구배경
- 1.2 연구 목적

2. 이론적 배경 및 관련 연구

- 2.1 하이브리드 AI시스템 개념
- 2.2 멀티 테넌트 시스템에서의 신뢰성 및 효율성
- 2.3 기존 접근 방식과 프레임워크 비교
- 2.4 멀티 에이전트 리스크와 하이브리드 구조의 필요성

CONTENTS (2/2)

3. 하이브리드 AI 에이전트 시스템 접근법

3.1 제안 배경 및 설계 목적

3.2 시스템 설계 원칙

3.3 데이터 보안 및 개인정보 보호 중심의 멀티테넌트 설계 (+ISMS-P 기준 부합성)

3.4 액션메소드 그룹 기반 처리 구조

3.5 하이브리드 AI 에이전트 처리 흐름

3.6 응답 조립 및 엔티티 상태 연동 구조

3.7 적용 시 기대 효과

4. 결론

5. 후속 연구

2

1. 서론 (1/2)

❖ 연구배경

(1) 규칙 기반 시스템: 높은 예측 가능성과 통제 가능성 제공하나, 유연성 부족

(2) LLM기반 시스템의 한계: 판단 오류와 응답 불확실성 위험 내포

(3) 멀티 에이전트 협업 과정 위험: 충돌, 정보노출, 역할 충돌

- 시스템 신뢰성과 안정성 저하

- 특히 각 에이전트가 고도로 자율적인 구조 가질 경우

· 통제되지 않은 상호작용이 발생하거나 시스템 전체의 예측 가능성 낮아짐

3

1. 서론 (2/2)

❖ 연구 목적

- (1) 규칙 기반 시스템과 LLM을 결합한 하이브리드 AI 에이전트 아키텍처 제안
- (2) 다음의 세가지 측면 실질적 개선 목표
 - 에이전트 시스템의 신뢰성 확보
 - 보안 설계 통해 보안성과 컴플라이언스 대응 능력 강화
 - 운영 안정성과 확장성 동시 달성
 - LLM 유연성 유지하면서 자원 효율성과 응답 일관성 고려한 실행 구조, 호출 최적화 전략

4

2. 이론적 배경 및 관련 연구

❖ 하이브리드 AI 시스템 개념

- (1) 개요: 규칙 기반 시스템과 LLM 결합 상황에 맞는 기술을 선택적으로 적용하는 구조
 - 규칙기반 시스템은 예측 가능한 질의에 대해 빠르고 정확한 응답 제공
 - LLM은 복잡한 질의나 추론이 필요한 작업을 처리하는데 유리
- (2) 하이브리드 구조는 효율성과 유연성을 동시 제공
 - 고비용 문제와 데이터 보안 문제 해결 방안 제시

❖ 멀티 테넌트 시스템에서의 신뢰성 및 효율성

- (1) 멀티테넌트 환경은 하나의 시스템이 복수의 테넌트를 위해 고객 데이터를 처리하는 구조
- (2) 보안 및 역할 기반 접근제어 중요
- (3) 각 테넌트의 데이터를 독립적으로 처리하고 보안 강화 및 자원 효율적 분배
 - 데이터 격리
 - 시스템 신뢰성과 효율성 보장 필요

5

2. 이론적 배경 및 관련 연구

❖ 기존 접근 방식과 프레임워크 비교

(1) 랭그래프

- 상태 기반 설계 프레임워크로, 대화 흐름을 명시적으로 구성할 수 있는 장점이 있으나, 규칙기반 대비, **실시간 구조 변경**, 병렬 처리 설정이 복잡해 유지보수와 **확장성에 한계** 존재함.

(2) Google (Gemini)

- 도구 통합과 대화형 처리에 강점이 있는 Google의 차세대 LLM으로 복잡한 질이 처리에 효과적이거나, **높은 운영비용과 응답 제어의 어려움**이 존재하여 실무에서는 규칙 기반 시스템과의 하이브리드 구성이 요구됨

(3) Anthropic (Claude)

- 헌법적 AI(Constitutional AI) 기반으로, 응답의 윤리성과 신뢰성을 확보하려는 시도를 구현 자기 검열 메커니즘은 존재하나, **보안성과 실행 제어에 한계**가 있어 기업 환경에서는 **규칙 기반 보안 설계가 필수적임**

6

2. 이론적 배경 및 관련 연구

❖ 멀티 에이전트 리스크와 하이브리드 구조의 필요성

(1) 자유성 높은 AI 에이전트가 협업하는 멀티에이전트 시스템 리스크

- **비의도적 상호작용, 실행 충돌, 신뢰도 저하**
 - 비의도적 상호작용: 의도하지 않은 메시지 오용
 - 실행 충돌: 리소스 경쟁으로 인한 Deadlock
 - 신뢰도 저하: 오작동한 에이전트의 영향이 전체 시스템으로 확산
- 특히 **에이전트 간 역할 분리 없이** 각자 자율적인 판단 수행시 **문제 발생**
 - 전체 시스템에서 **의도치 않은 충돌 및 효율 저하** 발생

(2) 단일 LLM기반 에이전트 구조로는 실시간 충돌이나 **다중 에이전트 간 실행 흐름 제어 어려움**

- 명시적이고 예측 가능한 규칙 기반 제어 구조의 결합이 필수적임.

7

3. 하이브리드 AI 에이전트 시스템 접근법

3.1 제안 배경 및 설계 목적

❖ 제안 배경

- (1) 산업 현장에서의 신뢰성, 보안성, 운영 효율성 확보라는 실질적 문제 해결 필요
- (2) **자율성 높은 AI 에이전트**는 다음과 같은 한계에 직면함
 - 응답 일관성 부족
 - 중요 정보 노출 위험
 - 자원 과다 사용

❖ 설계 목적

- (1) **신뢰성 강화**: LLM의 낮은 신뢰성을 규칙 기반 제어 구조로 보완, 명확한 실행 흐름 확보
- (2) **보안성 확보**: 데이터 교차 방지, 비식별화 처리 등 멀티 테넌트 기반 보안 설계
 - 개인정보 유출 최소화, 컴플라이언스 대응력 강화
- (3) **자원 효율성과 확장성 확보**
 - 창의적 언어 생성은 LLM에 맡기되, 명확하거나 반복적 작업은 규칙 기반 처리로 분산

8

3.2 시스템 설계 원칙

❖ 개요

- **하이브리드 AI 구조** 기반 규칙 기반 처리와 LLM 기반 생성 **병행**하는 복합 처리 구조

❖ 예측 가능한 작업은 규칙기반 처리

- 반복되는 질의 응답, 포맷 변환 등은 외부 설정 기반의 규칙으로 처리

❖ LLM은 **창의적 판단**(예 생성 작업)에 제한적 사용

- 텍스트 요약, 주제 추출, 고객 응대 문장 생성 등 창의적 작업에만 호출

❖ 자원 효율성 극대화

- 불필요한 LLM 호출 억제하여 처리 비용 절감 (**고비용 자원을 단순 처리에 사용하는 실수**가 발생함)

❖ 멀티 테넌트 보안 고려

- **테넌트별 데이터 격리, 권한제어**, 인증 구조 포함

9

3.3 데이터 보안 및 개인정보 보호 중심의 멀티테넌트 설계

3.3.1 데이터 파티셔닝과 테넌트 분리

- 테넌트 간 데이터 교차를 원천적 차단 (테넌트ID 기준으로 별도 처리)

3.3.2 개인정보 비식별화 및 응답 후처리

- 민감정보는 LLM 호출 전 엔티티명으로 치환

3.3.3 인증 및 권한 제어

- 역할 기반 권한 부여(RBAC) 및 JWT(JSON Web Token), 세션 등 표준 인증 방식 사용

3.3.4 참조 기반 민감정보 보호

- 비식별 API 참조 방식 채택

3.3.5 프롬프트 해킹 대응 방안

- 입력-응답 단계별 검증 체계를 통해 프롬프트 인젝션 등 공격에 대비

3.3.6 ISMS-P 기준 부합성 (ISMS-P: 개인 정보 보호 포함 정보보안 인증 기준, KISA(한국인터넷진흥원)에서 총괄 운영)

- 주요 보안 항목을 ISMS-P 인증 프레임워크에 따라 설계 및 점검

10

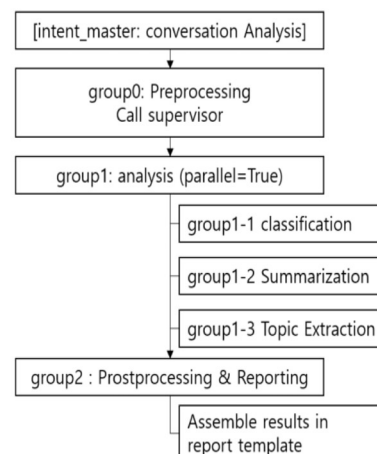
3.4 액션메소드 그룹 기반 처리 구조 (1/2)

❖ 개요

- 인텐트에 연결된 액션메소드 들은 그룹 단위로 직렬 또는 병렬 처리 구조
- 유연한 워크플로우 구성 가능

❖ 테이블 구조 및 정보 구조

- intent_master : 인텐트 정보
- action_method : 개별 액션 메소드
- action_method_group : 액션메소드 그룹
- intent_action_map : 인텐트와 액션메소드 매핑



<Figure 1> AI 에이전트의 워크플로우 구조 (상담분석 인텐트 예시)

11

3.4 액션메소드 그룹 기반 처리 구조 (2/2)

❖ LangGraph의 빌드 기반 워크플로우 구성과 한계

- 정적 구조 기반으로 설계된 워크플로우에는 적합하나,
- 실행 중 동적으로 구조를 변경하거나, 병렬 처리가 필요한 경우 워크플로우 구현의 난이도 상승

```
def build_workflow():  
    workflow = StateGraph()  
    workflow.add_node("step1", fn_step1)  
    workflow.add_node("step2", fn_step2)  
    workflow.set_entry_point("step1")  
    workflow.add_edge("step1", "step2")  
    return workflow.compile()
```

```
graph = build_workflow()  
graph.invoke({"input": "사용자 요청"})
```

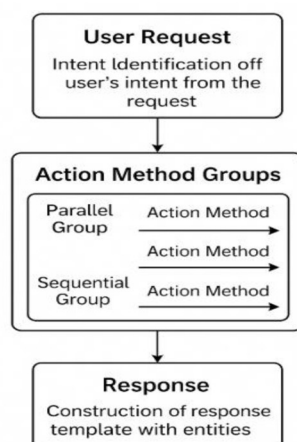
<Figure 2> LangGraph 기반 워크플로우 구성

12

3.5 하이브리드 에이전트 처리 흐름

❖ 하이브리드 에이전트 처리 흐름

- 인텐트 식별에서 기본처리는 직렬로 실행 함
- 액션메소드 그룹 호출에서 그룹내 처리 방식은 병렬 / 직렬 선택 가능



<Figure 3> 하이브리드 에이전트 처리 흐름도

13

3.6 응답 조립 및 엔티티 상태 연동 구조 (1/2)

❖ 응답 조립 및 엔티티 상태 연동 구조

- 각 액션메소드의 실행 결과가 사전에 정의된 엔티티 변수에 매핑
- 엔티티 상태 값을 기반으로 최종 응답 메시지 구성

액션메소드 명	기능 설명	반환 엔티티
classify_topic()	상담 주제 분류	[상담주제]
summarize_dialog()	대화 요약 생성	[상담요약]
extract_category()	고객 의도 분류	[상담분류]

<Table 2> 액션메소드별 반환엔티티

❖ 템플릿 기반 응답 조립

- 응답 메시지는 사전 정의된 템플릿에 따라 조립된다.

상담 내용은 다음과 같습니다.

상담 제목: [상담주제]

상담 분류: [상담분류]

상담 요약: [상담요약]

<Figure 6> 응답 구성 예시

14

3.6 응답 조립 및 엔티티 상태 연동 구조 (2/2)

❖ 구조적 장점 및 확장성

- 모듈화 된 분석 결과 관리
 - 각 분석 결과가 독립적으로 저장되고 관리되어 **디버깅 및 검증 용이**
- **재사용성과 유지보수성**
 - 응답 포맷 변경이나 기능 추가 시, 엔티티 매핑만 수정하면 됨. (**노코드 및 로우코드 환경에 적합함**)
- 템플릿 기반 자동화 가능
 - 다양한 고객 응대 **유형에 따라 템플릿만 추가**하면 빠르게 대응 가능
- 새로운 기능 확장 용이
 - 새로운 액션 메소드가 추가될 경우, 대응되는 **엔티티만 등록**하면 기존 응답 시스템과 손쉽게 연동

15

3.7 적용 시 기대효과

❖ 적용 시 기대효과

- 구조적 분석 가능성과 검증 용이성
 - 액션메소드와 엔티티 기반 응답 조립 구조로 디버깅과 검증 용이함
- 자원 최적화 및 병렬 처리 효율
 - 반복적이거나 명시적 작업은 규칙기반으로 LLM 호출을 생략하거나 간소화 하여 LLM 호출 비용 절감
- 다층 보안 체계 통한 데이터 보호 효과
 - 입출력 단계의 다층 검증 체계로 개인정보 보호 대응 강화
 - ISMS-P 기준에 따라 정보보호/개인정보보호 동시 대응
- 유연한 확장성과 유지보수 편의성
 - 템플릿 기반 응답 구조와 엔티티 중심 상태 관리로 기능 추가 시 수정 범위 최소화
 - 노코드/로우코드 환경과도 친화적 구조

16

4. 결론

❖ 개요

- 멀티 에이전트 환경에서의 실질적인 문제를 해결하기 위한 설계 전략

❖ 핵심 기여점 통해 산업 적용 가능성과 기술적 완성도 동시에 확보

- 하이브리드 설계 원칙의 체계화
- 다층 보안 구조의 통합
- 액션메소드 기반 실행 구조와 엔티티 중심 응답 조립 방식

17

5. 후속 연구

5.1 실증 실험 및 성능 평가

- 멀티 에이전트 환경에서의 실질적인 문제를 해결하기 위한 설계 전략

5.2 산업 현장 적용 가능성 검토

- 실제 고객 상담, 민원 처리, 내부 지원 자동화 등 다양한 산업 시나리오에 적용

5.3 자율 중심 에이전트 연구와의 비교 고찰

- 최근 클라우드 기반 생성형 AI 환경에서는 Planner 기반의 Autonomous AI agent 활발 연구
 - . 에이전트가 목표를 스스로 정의, 도구 선택하여 실행 하는 구조
 - . 높은 자율성과 유연성 제공
- 제안된 구조적 차이, 제어 가능성, 보안성 측면의 구체적 비교를 통해 산업 적용에 적합한 아키텍처 전략 고찰